3.4.2 Framework

BLOOM was trained using Megatron-DeepSpeed[20] (Smith et al., 2022), a framework for large-scale distributed training. It consists of two parts: Megatron-LM[21] (Shoeybi et al., 2019) provides the Transformer implementation, tensor parallelism, and data loading primitives, whereas DeepSpeed[22] (Rasley et al., 2020) provides the ZeRO optimizer, model pipelining, and general distributed training components. This framework allows us to train efficiently with *3D parallelism* (Narayanan et al., 2021, shown in Figure 6), a fusion of three complementary approaches to distributed training. These approaches are described below:

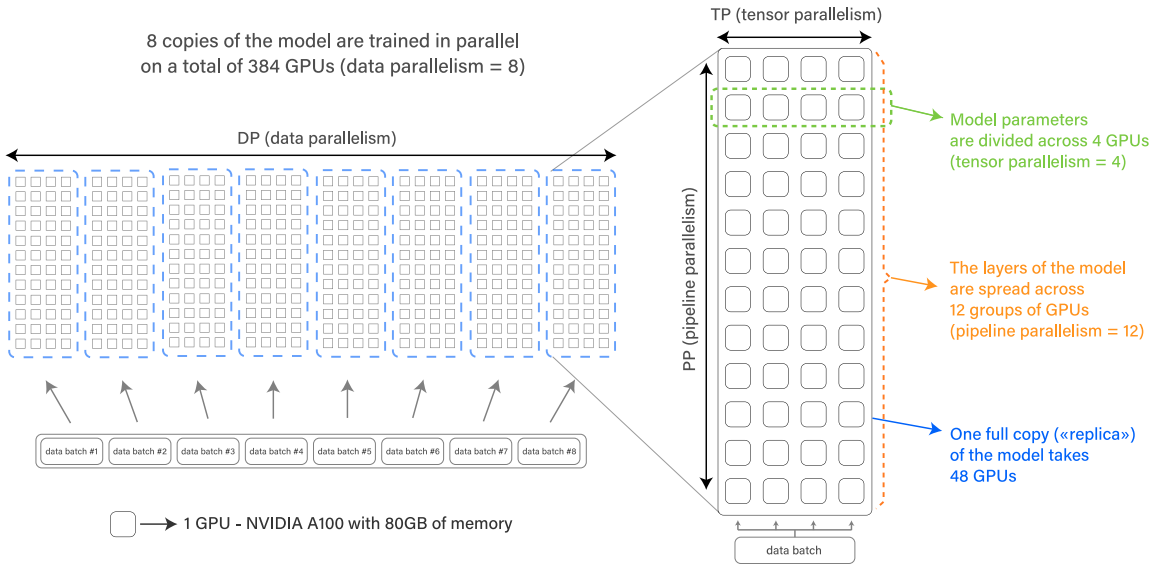

Figure 6: DP+PP+TP combination leads to 3D parallelism.

Data parallelism (DP) replicates the model multiple times, with each replica placed on a different device and fed a slice of the data. The processing is done in parallel and all model replicas are synchronized at the end of each training step.

Tensor parallelism (TP) partitions individual layers of the model across multiple devices. This way, instead of having the whole activation or gradient tensor reside on a single GPU, we place shards of this tensor on separate GPUs. This technique is sometimes called horizontal parallelism or intra-layer model parallelism.

Pipeline parallelism (PP) splits up the model's layers across multiple GPUs, so that only a fraction of the layers of the model are placed on each GPU. This is sometimes called vertical parallelism.

Finally, the Zero Redundancy Optimizer (ZeRO; Rajbhandari et al., 2020) allows different processes to only hold a fraction of data (parameters, gradients, and optimizer states)

20. `github.com/bigscience-workshop/Megatron-DeepSpeed`
21. `github.com/NVIDIA/Megatron-LM`
22. `github.com/microsoft/DeepSpeed`

required for a training step. We used ZeRO stage 1, meaning that only the optimizer states are sharded in this manner.

The four components described above are combined together to allow scaling to hundreds of GPUs with extremely high GPU utilization. We were able to achieve 156 TFLOPs in our fastest configuration with A100 GPUs, attaining our objective of half of the theoretical peak performance of 312 TFLOPs (in `float32` or `bfloat16`).

3.4.3 Floating Point Format

In earlier experiments with 104B-parameter models on NVIDIA V100 GPUs, we observed numerical instabilities that caused irreversible training divergences. We hypothesize that these instabilities stem from our initial use of IEEE `float16` — a 16-bit floating point format with a very limited dynamic range that can cause overflows. The NVIDIA A100 GPUs that we ultimately had access to support the `bfloat16` format (Wang and Kanwar, 2019; Kalamkar et al., 2019), which has the same dynamic range as `float32`. On the other hand, `bfloat16` still has much lower precision, which motivated our use of mixed-precision training (Micikevicius et al., 2018). This technique performs certain precision-sensitive operations such as gradient accumulation and softmax in `float32` precision and the rest of operations in lower precision, allowing us to achieve a balance of high performance and training stability. Ultimately, we performed final training in `bfloat16` mixed precision, which proved to solve the instability problem (in line with previous observation by Smith et al., 2022).

3.4.4 Fused CUDA Kernels

In general, GPUs cannot retrieve data to perform computations on and perform these computations at the same time. Moreover, the compute performance of modern GPUs is much higher than the speed of memory transfer required for every operation (often called *a kernel* in GPU programming). Kernel fusion (Wu et al., 2012) is an approach for optimizing GPU-based computations by performing several consecutive operations in only one kernel call. This approach offers a way to minimize data transfers: intermediary results stay in the GPU register instead of being copied into VRAM, saving overhead.

We used several custom fused CUDA kernels provided by Megatron-LM. First, we used an optimized kernel to perform LayerNorm, as well as kernels to fuse various combinations of the scaling, masking, and softmax operations. The addition of a bias term is also fused with the GeLU activation using the JIT functionality of PyTorch. As an example consequence of the use of fused kernels, adding the bias term in the GeLU operation adds no additional time, as the operation is memory-bound: the additional computation is negligible compared to data transfers between GPU VRAM and registers, so fusing both operations essentially halves their runtime.

3.4.5 Additional Challenges

Scaling to 384 GPUs required two final changes: disabling asynchronous CUDA kernel launches (for ease of debugging and to prevent deadlocks) and splitting parameter groups into smaller subgroups (to avoid excessive CPU memory allocations).

During training, we faced issues with hardware failures: on average, 1–2 GPU failures occurred each week. As backup nodes were available and automatically used, and checkpoints were saved every three hours, this did not affect training throughput significantly. A PyTorch deadlock bug in the data loader and disk space issues led to 5–10h downtimes. Given the relative sparsity of engineering issues, and since there was only one loss spike, which the model swiftly recovered from, human intervention was less necessary than in comparable projects (Zhang et al., 2022). Full details of our experience with training BLOOM and a detailed report of all issues we faced are publicly available.[23]

3.5 Training

Hyperparameter (↓)	BLOOM-560M	BLOOM-1.1B	BLOOM-1.7B	BLOOM-3B	BLOOM-7.1B	BLOOM
Architecture hyperparameters						
Parameters	559M	1,065M	1,722M	3,003M	7,069M	176,247M
Precision			`float16`			`bfloat16`
Layers	24	24	24	30	30	70
Hidden dim.	1024	1536	2048	2560	4096	14336
Attention heads	16	16	16	32	32	112
Vocab size			250,680			250,680
Sequence length			2048			2048
Activation			`GELU`			`GELU`
Position emb.			`Alibi`			`Alibi`
Tied emb.			`True`			`True`
Pretraining hyperparameters						
Global Batch Size	256	256	512	512	512	2048
Learning rate	3.0e-4	2.5e-4	2e-4	1.6e-4	1.2e-4	6e-5
Total tokens			341B			366B
Warmup tokens			375M			375M
Decay tokens			410B			410B
Decay style			`cosine`			`cosine`
Min. learning rate			1e-5			6e-6
Adam (β_1, β_2)			(0.9, 0.95)			(0.9, 0.95)
Weight decay			1e-1			1e-1
Gradient clipping			1.0			1.0
Multitask finetuning hyperparameters						
Global Batch Size	1024	1024	2048	2048	2048	2048
Learning rate	2.0e-5	2.0e-5	2.0e-5	2.0e-5	2.0e-5	2.0e-5
Total tokens			13B			13B
Warmup tokens			0			0
Decay style			`constant`			`constant`
Weight decay			1e-4			1e-4

Table 3: BLOOM & BLOOMZ Training Hyperparameters.

23. `github.com/bigscience-workshop/bigscience/blob/master/train/tr11-176B-ml/chronicles.md`